Senior Design Project: Spatio-Temporal Data Combination and Aggregation

Date: November 24th, 2025

Members: Michael Brown, Joe Crim, Jacob Rice

Advisor: Dr. Peter Jamieson

Table of Contents

1. **Executive Summary** **3**
2. **Motivation and Problem Statement** **3**
3. **System Architecture** **4**
    - 3.1 Data Specification
    - 3.2 Core Library Modules
    - 3.3 Project Objectives
4. **Testing Methodology and Validation** **8**
    - Simulation of Errors
5. **Results and Analysis** **9**
    - 5.1 Accuracy vs. Input Volume
    - 5.2 Confidence Metric Validation
6. **Conclusion** **11**
7. **Team Reflections** **12**
    - 7.1 Professional and Ethical Responsibility
    - 7.2 The Need for Engaging in Life-Long Learning
    - 7.3 Broad Education for Comprehending Global/Societal Issues
8. **References** **14**

1. Executive Summary

This report details the development of a Python-based software library designed to ingest, interpolate, and aggregate, or simply append, spatio-temporal tracking data from multiple distinct sources. Second, we describe the experiment to test our approach. The primary objective is to produce as close to a "Ground Truth" dataset as possible by statistically combining multiple imperfect input files and their associated measurements. While the initial development utilized Counter-Strike 2 (CS2) match data for validation, the architecture is designed for broad applicability in high-precision domains such as sports analytics (player tracking) and civil engineering (traffic flow analysis).

2. Motivation and Problem Statement

Spatio-temporal data is data containing both time information and location information. For this project, we consider data with a time granularity in seconds, and we measure two dimensional locations assuming a bird’s eye view. Two application examples of the use of this data include traffic flow and sports analytics. Civil engineers use spatio-temporal data sets for traffic pattern analysis and are starting to use more advanced sensors that provide more data, such as Lidar [2].

Player tracking in sports has exploded in popularity in recent years due to its ability to determine player performance and strategically advantageous field positions [1]. In both of these applications, the data collected from sensors is imperfect. More advanced sensors, such as Lidar and GPS, may not be available in actual sports matches or remote traffic intersections. Due to cost and flexibility limitations, cameras may be the only sensors available, but they are imprecise and can suffer from occlusion. Thus, our project combines spatio-temporal data sets

from multiple perspectives created for a system of tracking to create a dataset closer to the "Ground Truth," or what actually occurred. Our library can correct issues such as:

- **Temporal Gaps:** Missing timestamps due to occlusion or transmission loss.
- **Spatial Jitter:** Positional inaccuracy inherent to the tracking hardware or software.
- **Outliers:** Large deviations from the true position.

3. System Architecture

The system is modularised into distinct components handling input/output, core logic, and simulation.

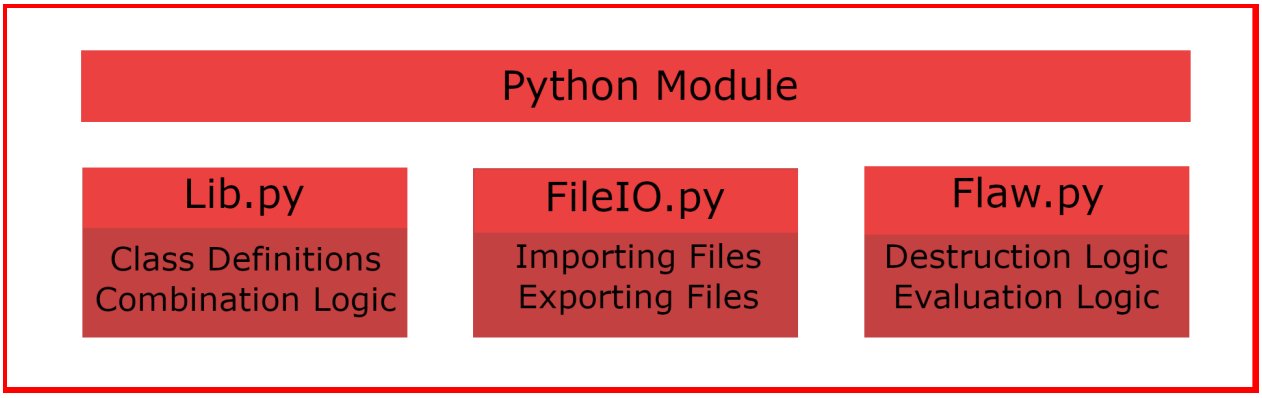


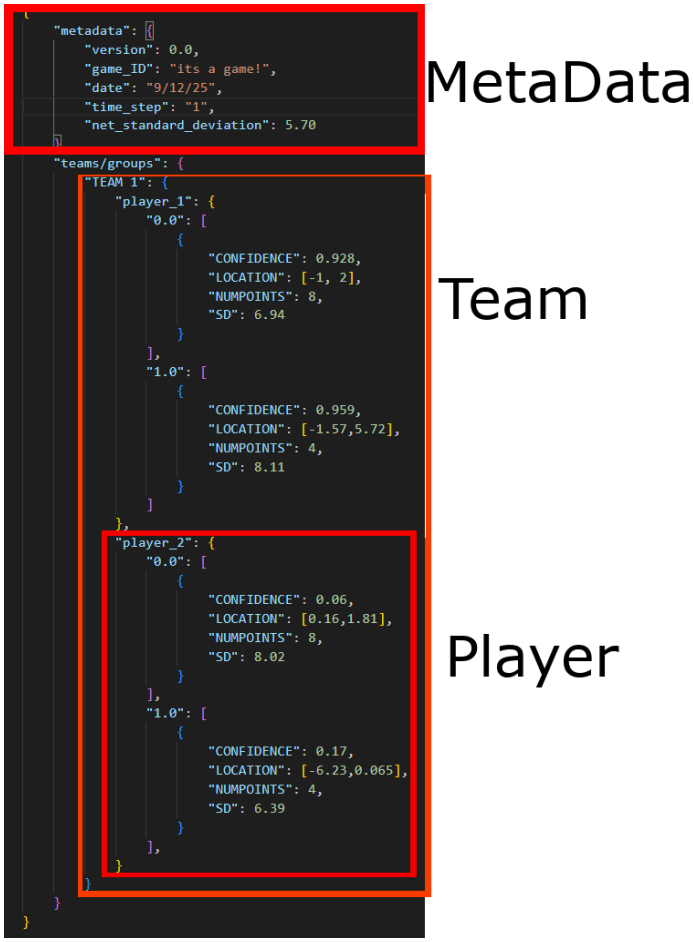
Figure 1. Umbrella diagram of Python Library.

3.1 Data Specification (`FORMAT.json`)

The system utilises a structured JSON data format designed to support easy parsing, readability, as well as allow for historic or non-historic "Compressed" files. Historic files contain all points from all input data sets, while non-historic or "Compressed" datasets only contain the single combined output data point for each timestep.

The format follows the following hierarchical structure:

- **Root Level:** Contains global metadata fields, such as the Game ID, Date, Time Step, and Net Standard Deviation.
- **Teams/Groups:** Represents the set of player groupings. Each team serves as a categorical container for all associated players; It is keyed by the name of the team/group.
- **Players:** Represents individual agents and their corresponding Time and Position Data.
- **Time Entries:** The fundamental unit of temporal data. Each entry includes:
    - **Location:** A two-element array specifying 2 spatial coordinates.
    - **Confidence:** Normalised metric between 0.0 and 1.0.
    - **Numpoints:** The number of source data points used to compute the location. This value serves as a damping factor to promote convergence toward the true position.
    - **Standard Deviation:** The standard deviation of the source points that make up the point, at the given timestep.




```
{
    "metadata": {
        "version": 0.0,
        "game_ID": "its a game!",
        "date": "9/12/25",
        "time_step": "1",
        "net_standard_deviation": 5.70
    }
    "teams/groups": {
        "TEAM 1": {
            "player_1": {
                "0.0": [
                    {
                        "CONFIDENCE": 0.928,
                        "LOCATION": [-1, 2],
                        "NUMPOINTS": 8,
                        "SD": 6.94
                    }
                ],
                "1.0": [
                    {
                        "CONFIDENCE": 0.959,
                        "LOCATION": [-1.57,5.72],
                        "NUMPOINTS": 4,
                        "SD": 8.11
                    }
                ]
            },
            "player_2": {
                "0.0": [
                    {
                        "CONFIDENCE": 0.06,
                        "LOCATION": [0.16,1.81],
                        "NUMPOINTS": 8,
                        "SD": 8.02
                    }
                ],
                "1.0": [
                    {
                        "CONFIDENCE": 0.17,
                        "LOCATION": [-6.23,0.065],
                        "NUMPOINTS": 4,
                        "SD": 6.39
                    }
                ],
            }
        }
    }
}
```


Figure 2. Breakdown of the JSON format.

3.2 Core Library Modules

The library uses two primary Python modules: one for data structures and one for analysis.

- **lib.py (Core Logic):** This file defines the object-oriented framework used throughout the system, including the Player, Team, and Teams_and_Meta classes. It also implements the major computational procedures for merging multiple data points:
    - **Weighted Arithmetic Combination:**

      Calculates a location at each time step by applying a confidence and point-weighted average:

$$\frac{w_1x_1+w_2x_2}{w_1+w_2}$$ where $w_i = Confidence \ * \ Number\ of\ Points$

This method is applied independently to both x and y coordinates.

Example

You have two locations:

- Point 1: (1,1), with 1 point used to calculate the location and confidence 0.5
- Point 2: (2,2), with 2 points used to calculate the location and confidence 0.75

Using these, the formulas become:

$w_1 = .5 \ * \ 1$ = .5 $\qquad$ $w_2 = .75 \ * \ 2 = 1.5$

$$\frac{.5 * x_1 + 1.5 * x_2}{.5 + 1.5} = \frac{.5+3}{2} = 1.75$$

This can be done similarly for y, since both points have identical x and y, and get a resultant location of (1.75,1.75).

- **Weighted Geometric Combination:**

Alternative merging strategy. Currently takes the weighted geometric mean of all the points of a given time, and averages the confidences. At this time, it only functions correctly with positive position values, as the geometric mean is only defined for positive numbers.

    - **Standard Deviation Calculation:** Computes the difference of coordinate estimates to quantify positional precision.
    - **Confidence Updating:** Adjusts the reliability score of a location by evaluating the similarity between contributing points. Similarity is computed using a modified cosine similarity to take into account distance as well as angle.
    - **Interpolation:** Detects missing timestamps and generates interpolated coordinates to maintain temporal continuity across the dataset.
- **fileIO.py (Input/Output Management):** This module serves as the interface for reading and writing to our file formats.

4. Testing Methodology and Validation

To evaluate the library's performance, we require a controlled testing environment where the "Ground Truth" is known. We utilize accurate match data from Counter-Strike 2 as our baseline “Ground Truth” since all Counter-Strike 2 (CS2) matches are available having tick level simulations from the server perspective. An online parsing utility and supporting Python scripts convert these matches into our proprietary JSON format.

4.1 Simulation of Errors (`flaw.py`)

Because real-world data errors are unpredictable, we created flaw.py. This script creates synthetic "bad" data from our perfect baseline by:

1. **Stochastic Removal:** Randomly deleting time entries to simulate sensor dropout.
2. **Noise Injection:** Artificially shifting X/Y coordinates to simulate sensor drift or calibration errors.
3. **Outlier Creation:** Shifting some points to simulate large sensor errors.

This allows us to feed flawed inputs into our experimental dataset and measure how closely the reconstructed output is to the original.

5. Results and Analysis

The performance of the library is evaluated based on spatial accuracy and the reliability of our calculated confidence metrics. The results are summarized in the graphs below.

5.1 Accuracy vs. Input Volume

Figure 1 shows a graph of the average error between the output data set and the “Ground Truth” for an increasing number of flawed input files. We use flaw.py to create an increasing number of flawed files off of a CS2 dataset, shown on the x-axis of Figure 1, we measure the error between the output data set and the “Ground Truth” from the original CS2 simulation file (shown on the y-axis of Figure 1). Every point in the flawed input data sets is moved up to 15 units (CS2 does not provide units in its location data) away from its true location and its confidence is adjusted to between 40% and 70% randomly. No data points are removed as the interpolation functionality of the library is still limited. Outliers are not included either as we want to see how the average error decreases without any large outliers causing substantial issues with the average error. This error is simply the average of the absolute value of the difference between each output point and the “Ground Truth” point.

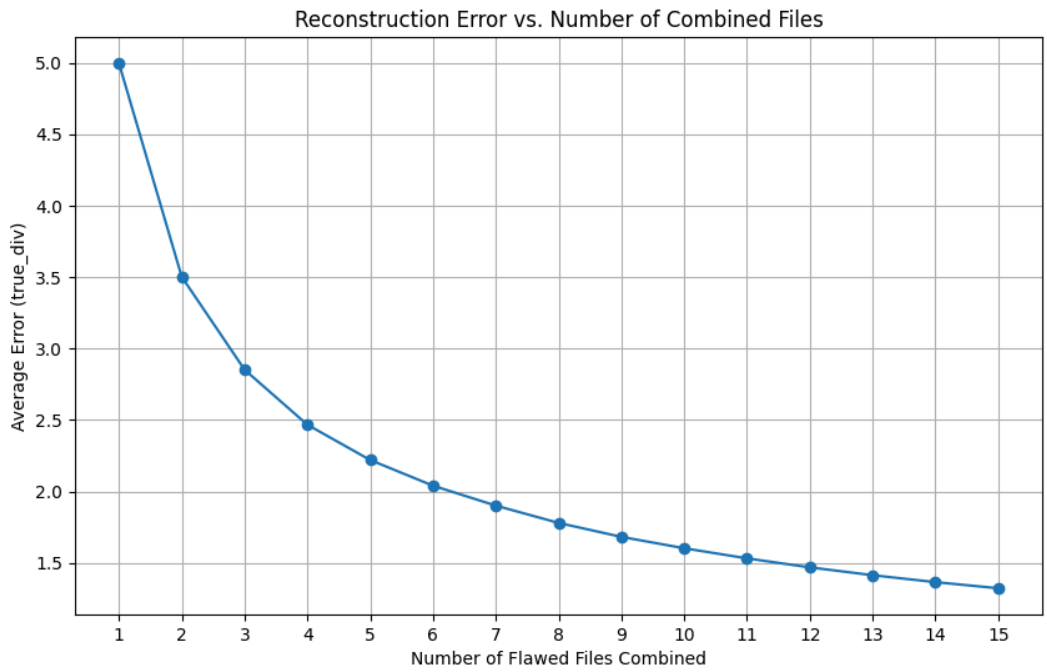


Figure 1: The true deviation from the “Ground Truth” of the output data set for an increasing number of input files.

As evident in Figure 1, the curve shows that as more files are inputted, our error decreases and approaches zero. After just four input files are combined, the average error from the true location is halved when compared to one data set. This demonstrates the effectiveness of our combination methodology when using the weighted arithmetic mean method as discussed in section 3.2.

5.2 Confidence Metric Validation

Figure 2 shows the evaluation of the internal logic of our confidence metric against the calculated standard deviation. The number of input files with the same creation parameters as in Figure 1 is shown on the x-axis. The y-axis shows the resulting output confidence for the blue curve, and the percent of points (as a decimal) within one averaged standard deviation (this is

an average of the standard deviations calculated for each individual output data point) for the orange curve. That is why it is labeled Value / Percentage. We consider a “Ground Truth” point to be contained within one standard deviation of an output point if the “Ground Truth” point is within a circle with a radius of the average standard deviation centered on the output point.

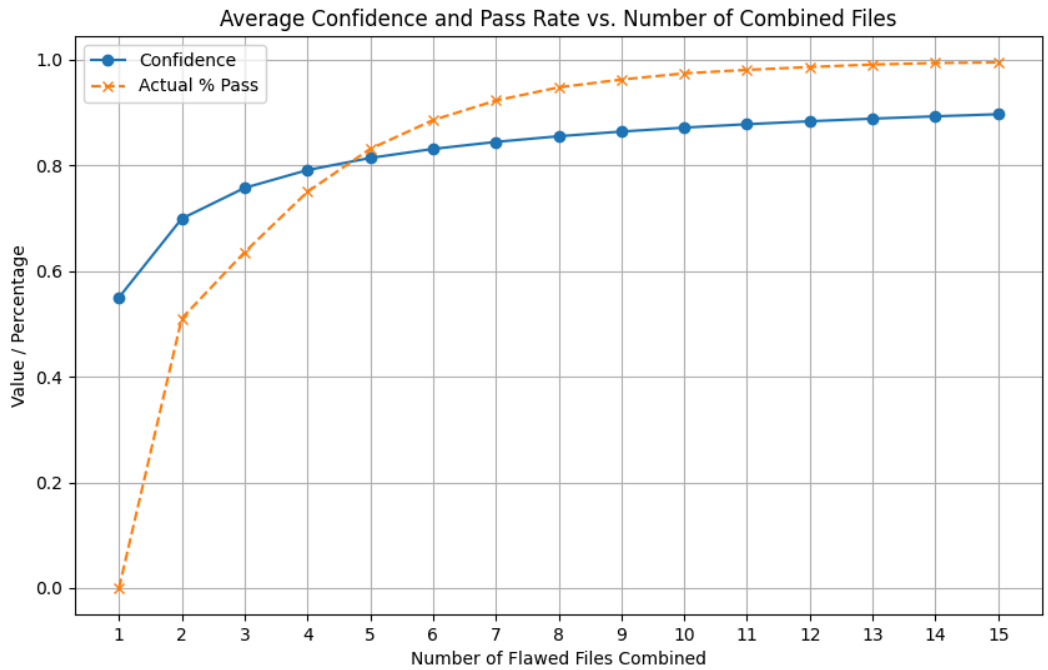


Figure 2: The percent of points contained within one average standard deviation of the output data set compared to the average confidence in the output data set.

The results show that after four input files, our average confidence is slightly lower than the true percentage of “Ground Truth” points captured within a circle having a radius of one average standard deviation from the points in the output data set. While we aim to align these values closer—so that confidence accurately reflects the likelihood of a “Ground Truth” point being within one standard deviation of an output point—this is a positive outcome. It indicates that

after a brief initialization period (four files), our confidence measurement provides a functional, albeit conservative, estimate of accuracy.

6. Conclusion

Our Senior Design Project of a Spatio-Temporal Python Library, so far, successfully aggregates multiple noisy datasets using statistical merging and interpolation to yield an output file superior to any single input. The output file approaches the “Ground Truth” as more files are inputted, and the confidence of each point meaningfully indicates the likelihood of the true point being contained within a radius the size of one average standard deviation from the output point. Next semester, we intend to add more methods of combination, improve our interpolation implementation for missing points, and align the average confidence and likelihood of a point being within one standard deviation better.

7. Team Reflections

This design project included software design, data analysis, and understanding the broader context and responsibilities of engineering.

7.1 Professional and Ethical Responsibility

As engineers, our responsibility extends beyond just making code that "works." We must also consider *how* it works and *how* it will be used. This project highlighted several ethical considerations:

- **Data Integrity:** The entire purpose of this library is to improve data integrity. In a professional setting, presenting data without an understanding of its quality, confidence,

and limitations is irresponsible. Our library, with its built-in confidence and deviation metrics and evaluation tools, promotes a culture of being transparent about data quality.

- **Impact on Individuals:** The data we are modeling represents *people* in the case of sports analysis. Decisions based on this data—from a player's contract value to their training regimen—have real-world consequences. Ensuring the data is as accurate as possible, which our library helps to do, is an ethical imperative.

7.2 The Need for Engaging in Life-Long Learning

This project was a powerful reminder that our education does not start or end with a degree.

- **Evolving Domains:** The *types* of errors we model in flaw.py are based on today's sensor technology (network drops, GPS jitter). As technology changes (e.g., new AI-based optical tracking), new and more subtle error modes will emerge (e.g., player misidentification). We must engage in life-long learning about the *application domains* we work in, not just about the code itself.
- **Evolving Algorithms:** We implemented arithmetic and geometric means. These are statistically simple. The fields of sensor fusion, statistics, and machine learning are vast and evolving daily. To truly improve, we need to continuously study new papers and learn new, more complex algorithms like Kalman filters and Bayesian inference [3]. We considered the SLAM algorithm when designing the library as our library uses mapping concepts related to SLAM. We update locations as new information either supports or contradicts what we are measuring [4].

7.3 Broad Education for Comprehending Global/Societal Issues

This project, which at first glance seems like a niche tool for sports analytics, is actually a microcosm of a critical engineering challenge. A broad education is what allows us to see this connection.

- **The "Flawed Data" Problem is Universal:** Our library's logic is not limited to sports. The same principles apply to:
    - **Autonomous Vehicles:** Fusing data from LiDAR, radar, cameras, and GPS. A "flaw" (a misread stop sign) has life-or-death consequences.
    - **Climate Science:** Fusing data from thousands of disparate sources (satellites, weather balloons, ocean buoys) to build a model of global climate. The "reconstruction" of this data directly impacts global policy.
    - **Public Health:** Fusing case reports from different hospitals, cities, and countries to track a pandemic, where each source has different testing standards and reporting delays.
- **Our Solution's Impact:** An engineering solution for one domain can have a massive impact on another. The code and principles from our "player tracker" could be adapted to improve the accuracy of a self-driving car's perception system or a climate scientist's model. Understanding this "horizontal" impact is impossible with a narrow, code-only education. We must understand the statistics, physics, and societal contexts in which our code will operate. This project has reinforced our understanding that engineering solutions are never created in a vacuum; they are deeply embedded in, and have the power to reshape, our societal fabric.

8. References

[1] D. R. Pintella *et al.*, "Player Tracking Data in Sports," *Annu. Rev. Stat. Appl.*, vol. 10, Art. no. 1, 2023.

[2] ASCE, "Lidar Sensors Are Paving the Way to More Connected and Safer Roads," *Civil Eng. Source*, [Online]. Available: https://www.asce.org/publications-and-news/civil-engineering-source/article/2025/07/14/lidar-sensors-are-paving-the-way-to-more-connected-and-safer-roads. Accessed on: October. 25, 2025.

[3] MathWorks, "Linear Kalman Filters," *MATLAB Documentation*, [Online]. Available: https://www.mathworks.com/help/fusion/ug/linear-kalman-filters.html. Accessed on: December. 5, 2025.

[4] MathWorks, "What Is SLAM (Simultaneous Localization and Mapping)," *MATLAB Documentation*, [Online]. Available: www.mathworks.com/discovery/slam.html. Accessed on: December. 5, 2025.